

CLAUDE.md — Working rules for this repository

Project: DRISHTI-BOP — AI video analytics for border surveillance on existing CCTV

Spec: `docs/BUILD_SPEC.md` — read it before writing code. It is authoritative. **Context:** Smart India Hackathon 2026, PS 26187, Team SW-73 (ByteForge). Hackathon timeline: optimise for a working, demoable system over completeness.

Read this first

1. `docs/BUILD_SPEC.md` §3 (architecture), §6 (data model), §7 (module contracts), §16 (build phases).
 2. Build in the phase order of §16. Do not start a phase until the previous phase's acceptance criteria pass.
 3. Interface contracts in §7 are frozen. If one needs changing, say so explicitly and update the spec in the same commit.
-

Working rules

Scope

- Implement exactly what the current phase asks. Do not build ahead into later phases or [STRETCH] items.
- If a task is ambiguous, state the assumption at the top of your response and proceed. Do not stall on clarification for small things.
- Anything marked [DEMO-CRITICAL] gets the careful version. Anything marked [STRETCH] gets skipped unless explicitly requested.

Code

- Python 3.11. Type hints everywhere. `ruff` + `black` (line length 100). `mypy` on `worker/` and `api/` in non-strict mode.
- Dataclasses (frozen where they represent values) for internal types; Pydantic v2 for anything crossing a process boundary.
- Async for I/O in the API layer. The worker's hot path is threaded, not async — do not mix.
- No bare `except:`. No silent failures. Every caught exception logs with context.
- Config over constants. If a number could plausibly need tuning at a BOP, it belongs in `config/`, not in the source.
- TypeScript strict mode in the dashboard. No `any`.

Testing

- Every module in `worker/src/drishti_worker/` needs unit tests before it is considered done.

- Geometry, risk scoring, Merkle proofs, and plate validation get property-based tests (`hypothesis`) — they are pure functions with sharp edge cases.
- Integration tests run against fixture RTSP streams served by MediaMTX from local MP4s, never against a live camera.
- Do not mock the thing under test. Mock the GPU, mock Fabric, mock the network — not the logic.

Commits

- Conventional commits: `feat(evqm): add dark channel prior fog metric`.
 - One logical change per commit. A commit that touches five modules is five commits.
-

Things that will break the demo — treat as blockers

1. **Alert spam.** No debounce → a person standing on a tripwire generates dozens of alerts per minute. Debouncing, correlation, and `min_track_age` are Phase 3 requirements, not polish.
 2. **RTSP stalls.** OpenCV's `VideoCapture` hangs silently on dropped RTSP. The watchdog + reconnect in §7.1 is mandatory from Phase 1.
 3. **TensorRT cold start.** First inference takes ~2 s. Warm up with 10 dummy inferences at startup or the demo looks frozen.
 4. **Browser cannot play RTSP.** Video goes through MediaMTX → HLS. Test this early; it surprises people on demo day.
 5. **EVQM flapping.** Without hysteresis, a passing headlight flips the processing profile every second. `enter_samples=3`, `exit_samples=5`.
 6. **Fabric setup.** Highest-variance task in the project. Build the mock ledger backend first and wire everything to the interface. Fabric never blocks the critical path.
 7. **Evidence hashing.** Canonicalise (JCS, RFC 8785) before hashing, exclude `evidence_hash` and `ledger` fields, hash in the worker at assembly time. Get this wrong and verification silently fails later, which is worse than failing loudly.
-

Invariants — do not violate these without explicit discussion

Invariant	Why
Evidence snapshots and clips are original, unenhanced frames	Enhancement is for the model; evidence is for the court. Store enhancement params as metadata instead.
Every alert carries reason codes and an exact risk breakdown	Principle P2. An alert that cannot be explained must not be raised.
Risk contributions sum exactly to the risk score	The model is additive on purpose. If they don't sum, the explanation is a lie.
Face analytics defaults to disabled	Principle P6, and it is the answer to the privacy question.
Face embeddings from non-matching tracks are discarded at track close	No silent enrolment, ever.
Plate text is stored as HMAC outside of fired-alert evidence	A database leak must not be identifying.
Recording continues if analytics crash	Principle P8. Never lose video because AI broke.
No automated response action	Human adjudicates. The system recommends.
Detections are always mapped back to original frame coordinates	Inference may run on resized/enhanced/tiled images. Carry the transform.
Ledger unavailability never blocks alerting	Queue the anchor, fire the alert.

Common tasks

```
bash
```

```
make up          # start infra (postgres, minio, redis, mediamtx)
make migrate     # alembic upgrade head
make seed        # demo sites, cameras, zones, users, watchlist
make worker      # run analytics worker against fixture streams
make api         # run FastAPI with reload
make dash        # vite dev server
make test        # pytest + vitest
make bench       # throughput/latency benchmark, writes to docs/BENCH.md
make eval        # run the evaluation set, writes metrics table
make demo        # everything, cold start, offline – the demo-day command
make fabric-up   # Fabric test network + chaincode deploy
```

When you are unsure

- **Performance vs. clarity** → clarity, except inside the per-frame hot path (§7.3–7.5), where performance wins.
- **Accuracy vs. false alarms** → false alarms. An operator who stops trusting the system has zero recall. Raise thresholds, add evidence requirements, tighten debounce.
- **Feature vs. reliability** → reliability. A demo that works with five features beats one that crashes with twelve.
- **Building vs. asking** → build, and state the assumption. But if the decision changes a frozen interface contract or a privacy invariant, ask first.